Propensão de compra em e-commerce

Pipeline de MLOps reprodutível

FIAP Pós MLE · Tech Challenge Fase 2

Problema e dados

- Objetivo de negócio: identificar, pela navegação, as sessões com propensão de compra
- Dataset: Online Shoppers Purchasing Intention (UCI)
- Variáveis: páginas e tempo por tipo de página, taxas de rejeição e de saída, mês e tipo de visitante
- Alvo binário Revenue: a sessão terminou em compra?

Característica	Valor
Sessões	12.330
Variáveis de entrada	17
Sem compra (classe 0)	10.422 (84,5%)
Com compra (classe 1, positiva)	1.908 (15,5%)

Métrica: PR-AUC em vez de accuracy

O desbalanceamento (15,5% de compras) influencia a escolha da métrica. A PR-AUC avalia a classe positiva, quem compra, pelo equilíbrio entre precision e recall:

- foca em quem compra, a classe rara
- ignora o grande número de negativos
- mede o quão bem o modelo ranqueia os compradores
- não depende de um threshold

Classificador	Accuracy	PR-AUC
Sempre responde “não compra”	84,5%	0,156
HistGradientBoosting (validação cruzada)	–	0,753

Escopo: engenharia de ML, não o modelo

Notebooks: decisões

- EDA: entender os dados
- Modelagem: escolher modelo, métrica e threshold; as decisões viram o params.yaml
- Threshold 0,73: maximiza o F1 em validação cruzada no treino; o teste só reporta

Modelo (validação cruzada)	PR-AUC
HistGradientBoosting (escolhido)	0,7534
XGBoost	0,7532
Random Forest	0,7527
Regressão logística	0,663

Empate técnico entre os de árvore; escolhido por ser nativo do Scikit-Learn, com menos dependências que o XGBoost.

Pipeline: a entrega

- código limpo e testado
- dependências fixadas (uv.lock)
- dado e pipeline versionados (DVC)
- experimentos e modelos registrados (MLflow)
- tudo em containers (Docker)

Adicional

- Adicional: serviço de inferência sobre o modelo registrado, para fechar o fluxo inteiro

Arquitetura do pipeline



1. Dado e pipeline (DVC)

- dataset em duas versões: v1 (fev a out) e v2 (completo)
- preprocess limpa e divide em treino e teste (80/20)

2. Docker

- trainer é um job: baixa o dado, treina e termina
- inferência com imagem própria, separada do trainer

3. MLflow

- cada treino: um run e uma versão, com hash do dado e commit
- portão: promove só se superar o champion no mesmo teste

Obs.: por ora, tudo em um único repositório, para simplificar; em produção, serviços e pipelines separados.

Estrutura do código

Pacote	Papel	Classes essenciais
data	ler, limpar e dividir o dado	DataLoader: lê o CSV e valida as colunas · DataCleaner: remove duplicatas · DataSplitter: divisão estratificada, com seed fixa
features	pré-processar	PreprocessorFactory: monta o transformador de colunas, que vai dentro do Pipeline do modelo
training	treinar, registrar e promover	Evaluator: validação cruzada e métricas no teste · ExperimentTracker: runs, métricas e modelo no MLflow · ModelRegistry: versões e alias champion · PromotionGate: decide a promoção · Trainer: orquestra o treino
pipelines	estágios do DVC	preprocess e train: pontos de entrada de cada estágio
serving	inferência (adicional)	ModelLoader: carrega a versão do Registry · Predictor: probabilidade e classe pelo threshold · API (FastAPI): /health, /model e /predict

Boas práticas: testes automatizados, docstrings e type hints em todo o código, funções curtas.

Configuração e regra de promoção

dvc.yaml (estágios preprocess e train)

```
preprocess:
  cmd: python -m ...pipelines.preprocess
  deps: [data/raw/...csv, src/...]
  params: [seed, split]
  outs: [train.csv, test.csv]
train:
  cmd: python -m ...pipelines.train
  deps: [train.csv, test.csv, src/...]
  params: [seed, train, model]
  metrics: [metrics.json]
```

params.yaml (trecho)

```
seed: 42
train:
  primary_metric: average_precision
  threshold: 0.73
model:
  learning_rate: 0.05
```

PromotionGate (simplificado)

```
def decide(candidate, champion):
    if champion is None:
        return promote("primeiro modelo")
    if candidate > champion:
        return promote("supera o champion")
    return keep("não supera o champion")
```

- **dvc.yaml:** estágios, dependências, parâmetros e saídas; o DVC só refaz o que mudou
- **params.yaml:** seed, split e hiperparâmetros num só lugar (decisões dos notebooks)
- **PromotionGate:** promove só se superar o champion, avaliado no mesmo teste

Demonstração: sequência de comandos

```
cat data/raw/online_shoppers_intention.csv.dvc   # ponteiro do dado: só um hash (v1)
make train                                       # 1ª execução: a versão 1 vira champion
make train                                       # 2ª execução: empate, o champion não muda
make data-v2                                     # dado novo (versão 2 do dataset)
cat data/raw/online_shoppers_intention.csv.dvc   # o ponteiro mudou de hash (v2)
uv run dvc push                                  # envia o dado novo ao bucket
make train                                       # retreino: a versão 3 supera o champion
make deploy-inference MODEL=1                    # API servindo a versão 1
make deploy-inference                            # API passa a servir o champion
```

Ferramenta	Endereço	O que observar
Console RustFS	localhost:9001	bucket dvc-remote: objeto com o nome do hash
MLflow	localhost:5001	Models: vazio; no fim, v1, v2 e v3 (champion na 3)
Swagger	localhost:8000/docs	sessão visitante_recorrente (fronteira), threshold 0,73: v1 → 0,60 (classe 0); champion → 0,75 (classe 1)

Limitações e próximos passos

Área	Hoje	Próximo passo
Retreino	disparado à mão, trocando o ponteiro do dado	gatilho automático
Dados	validação mínima: só confere as colunas	validação robusta: tipos, faixas, categorias e distribuição
Infra	local, com Docker Compose; sem deploy em nuvem nem CI	Kubernetes: trainer como job agendado

Reprodutibilidade: reproduzir.sh

```
./reproduzir.sh
```

1. Reseta o estado local do projeto (volumes, cache e dados gerados)
2. Instala as dependências nas versões exatas do uv.lock; roda lint e testes
3. Sobe o Docker Compose (S3 e MLflow) e treina duas vezes no container: o empate não muda o champion
4. Dado novo, retreino e promoção
5. Inferência: compara as duas versões

Em cada etapa, confere o resultado esperado. Por exemplo: lint e testes passando, PR-AUC igual ao valor esperado, empate sem trocar o champion, versão nova promovida e API respondendo.

Criado para quem avalia validar o pipeline com facilidade; também serviu para validar o fluxo durante o desenvolvimento.

Obs.: guia passo a passo em docs/reproducao.md; repositório: github.com/paulooorg/pos-mle-tech-challenge-2